Algorithmic Complexity

Y.N. Srikant

Visiting Professor
Chanakya University
(Formerly) Professor
Computer Science and Automation
Indian Institute of Science
Bangalore 560 012

Algorithms(1)

- Informally, an *algorithm* is a procedure to accomplish a specific job.
- A computer program is an *implementation* of an algorithm.
- Further, an algorithm must have a finite number of steps. This number may be dependent on the input size.
  - Implying that the corresponding program will HALT in a finite amount of time for all inputs. The exact amount of time may be dependent on input size.
- Obviously, we need *efficient* algorithms, which lead to efficient programs.

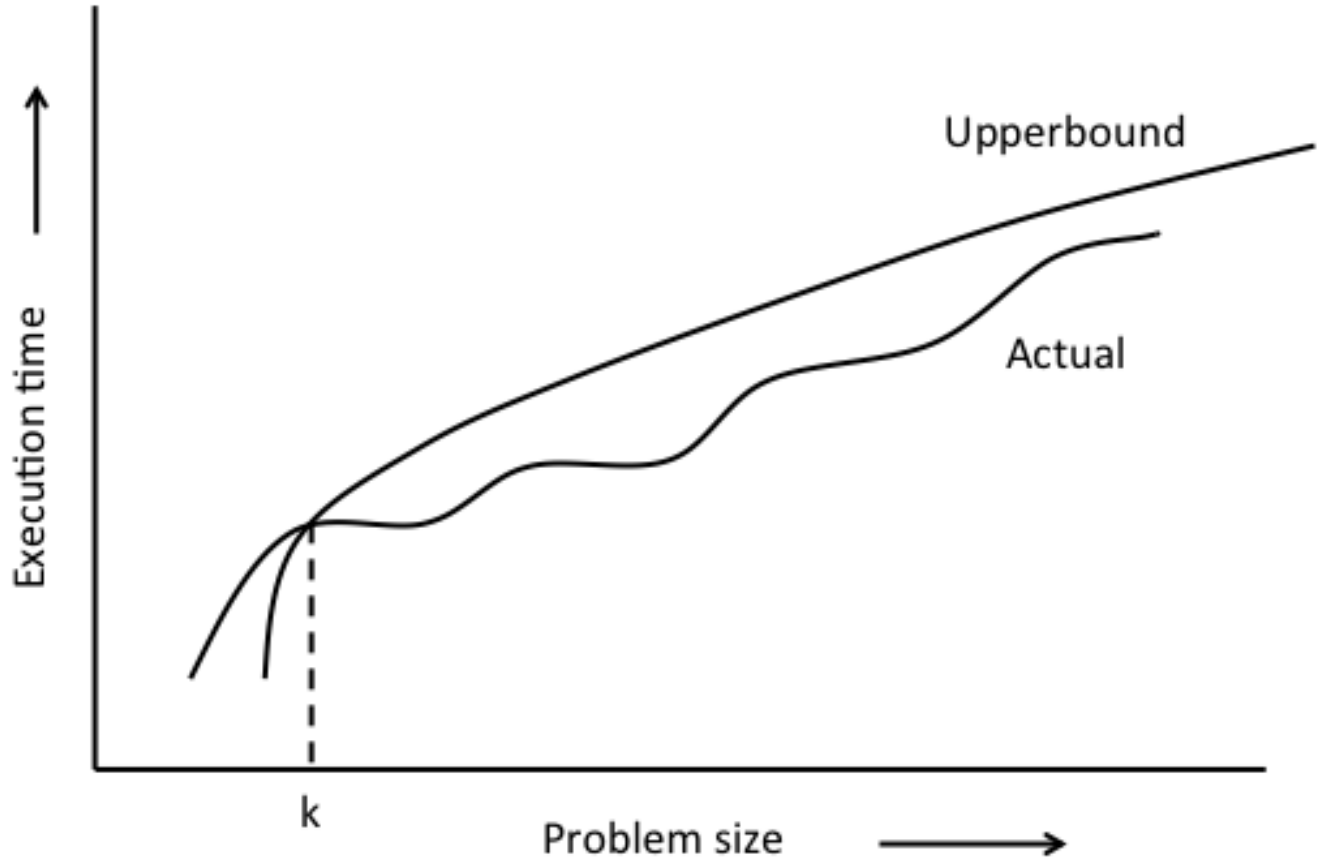
Algorithms(2)

- How do we measure the efficiency of algorithms?
  - The execution time expressed as a function of the input size is a good measure.
  - Power dissipated, energy consumed, and memory required are other measures.
  - In this course, we will concentrate on time and space.
- We need a model of computation in which timing information is also defined.
- The model of computation should be preferrably language- and machine-independent.

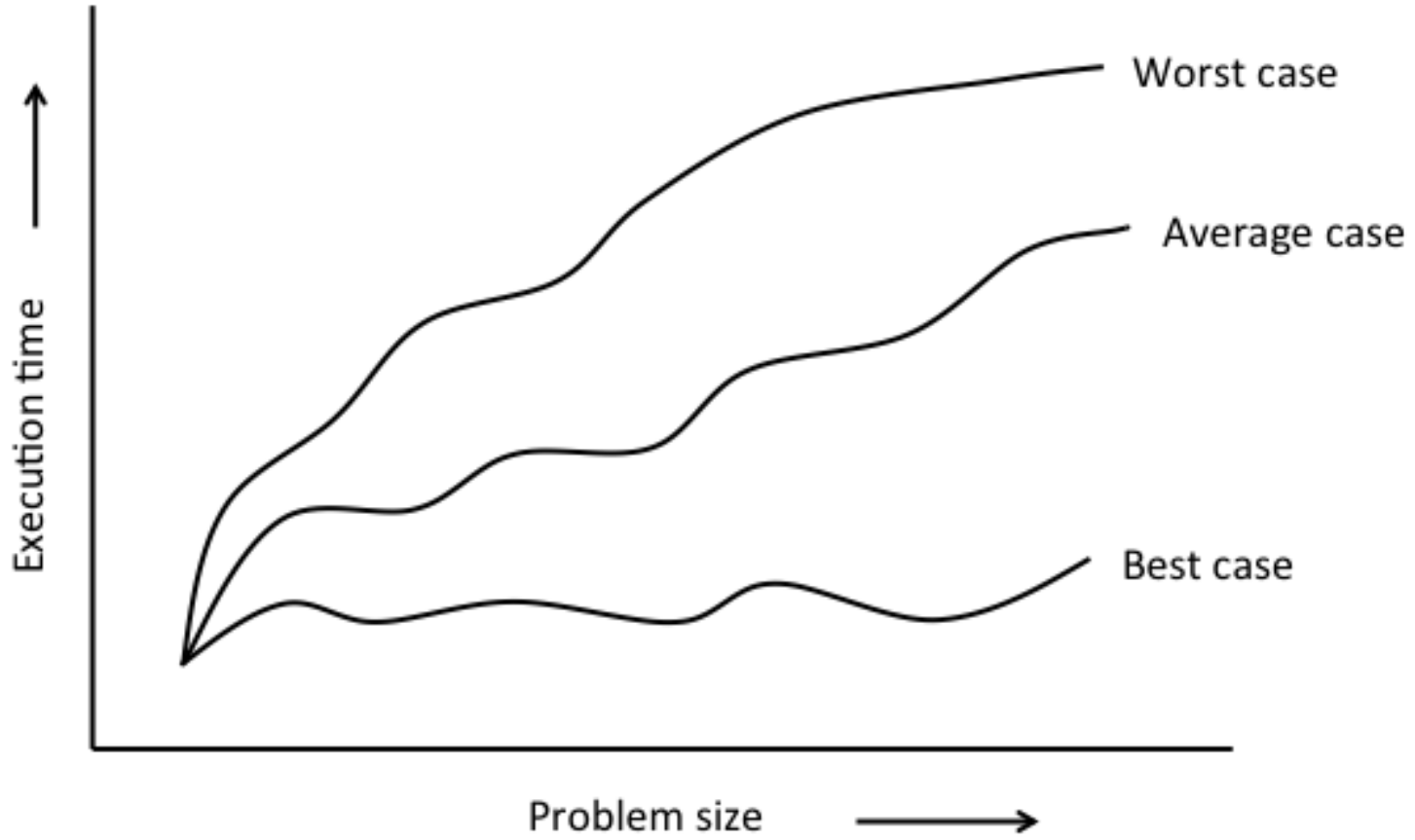
Time Complexity of Algorithms

- Exact functions that express the execution time of algorithms as a function of input size can be quite complex.
- Example: $f(n) = 2n^2 + 5log_2 n - 8$
- The behaviour of an algorithm cannot be easily inferred from such functions.
- We need to extract the “essential” behaviour from such functions.
- We need such a mechanism in order to distinguish between good algorithms and bad algorithms (with respect to time).

Upper Bound



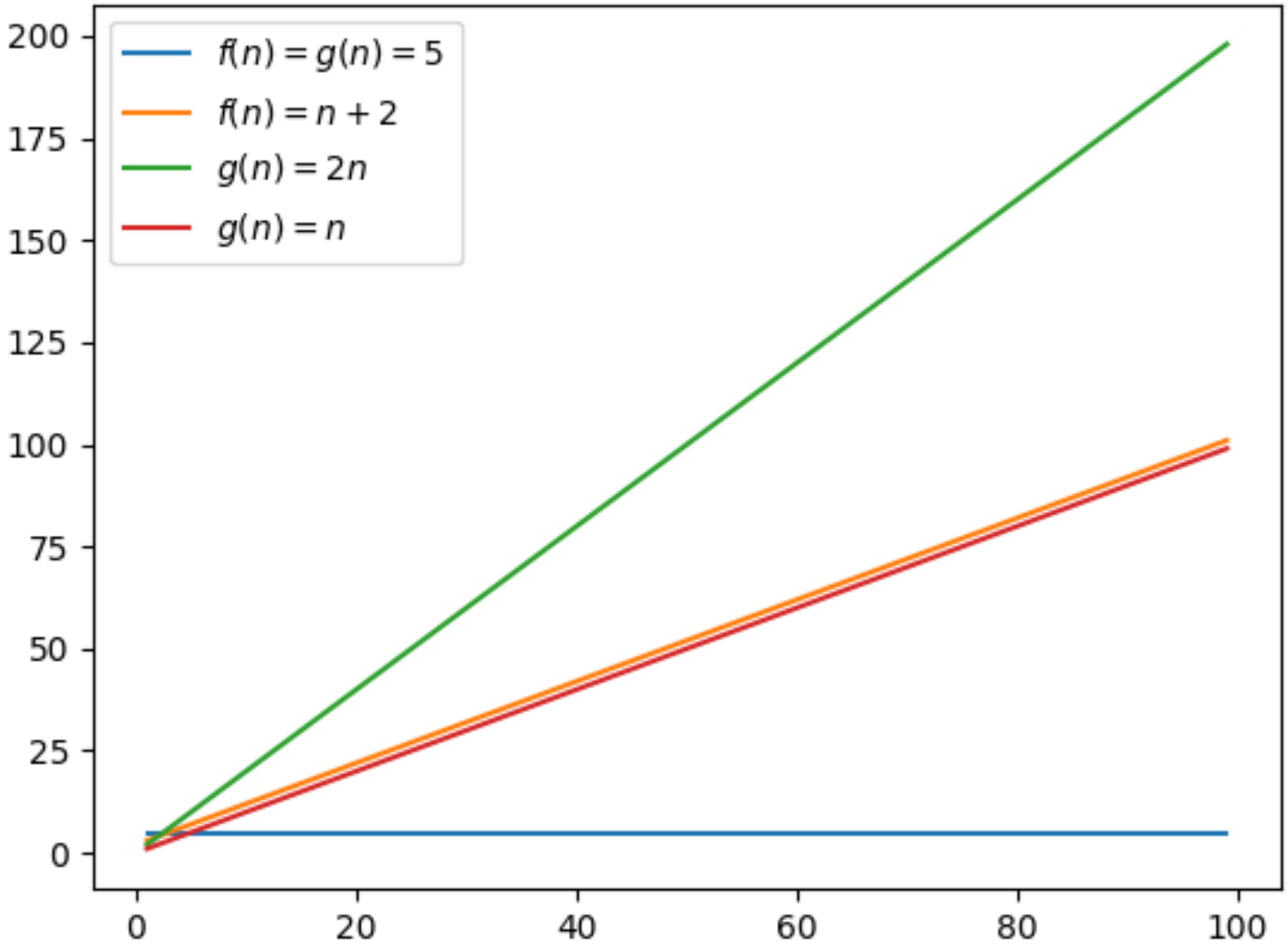
Best Case, Average Case, and Worst Case Complexity of an Algorithm



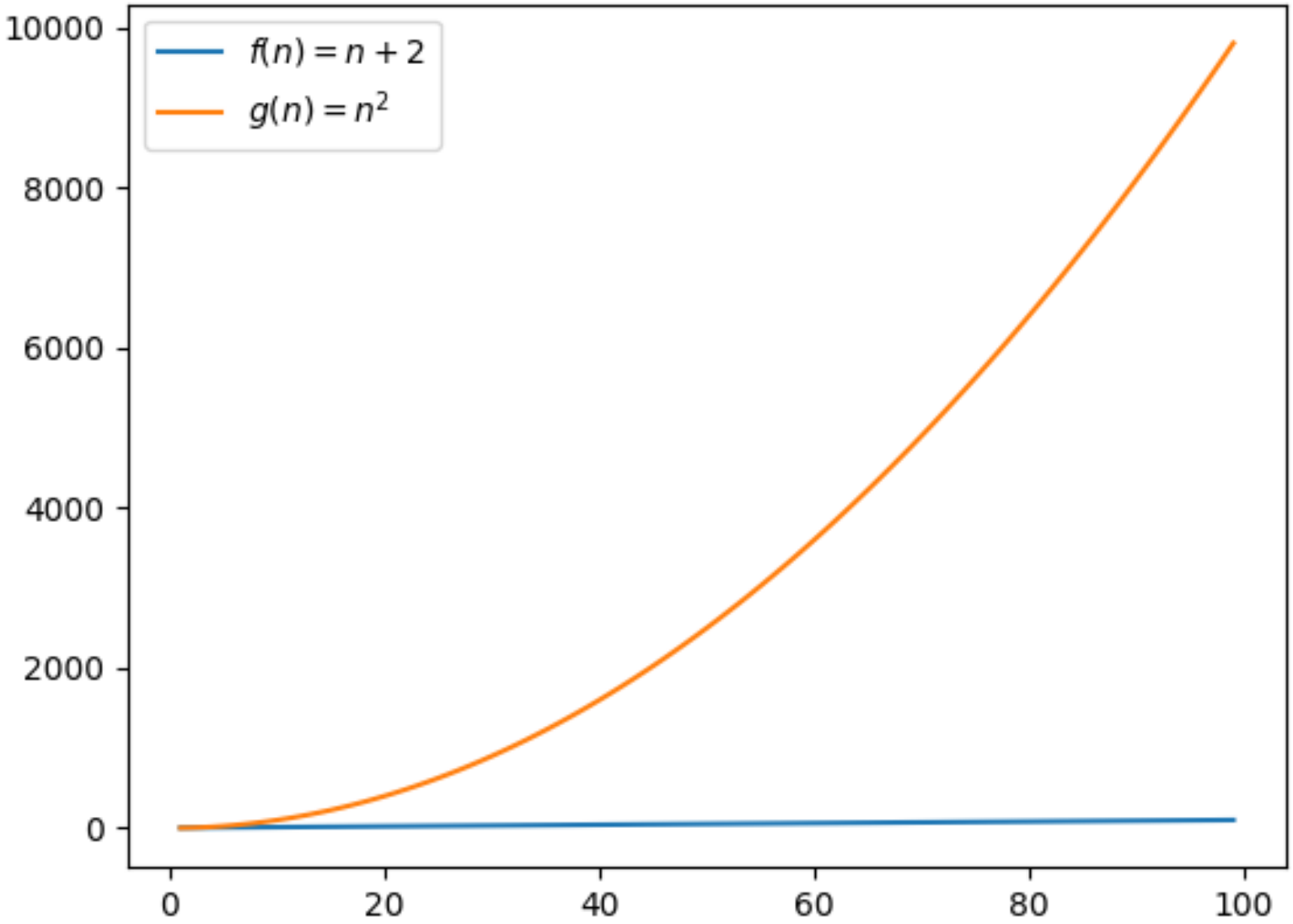
Time Complexity of Algorithms - Big 'oh' Notation(1)

- $f(n)$ is $O(g(n))$ iff there exist positive constants c and k such that $f(n) \leq c.g(n)$, for all $n \geq k$.
- Examples
  - $f(n) = 5$ is $O(1)$, because $5 \leq 5.1$, for all $n \geq 1$ $(c = 5,\ k = 1,\ g(n) = 1)$.
  - $f(n) = n + 2$ is $O(n)$, because $n + 2 \leq 2n$, for all $n \geq 2$ $(c = 2,\ k = 2,\ g(n) = n)$.
  - $f(n) = n + 2$ is also $O(n^2)$, because $n + 2 \leq n^2$, for all $n \geq 2$ $(c = 1,\ k = 2,\ g(n) = n^2)$.

Big ‘oh’ Example - 1 (plots using matplotlib)



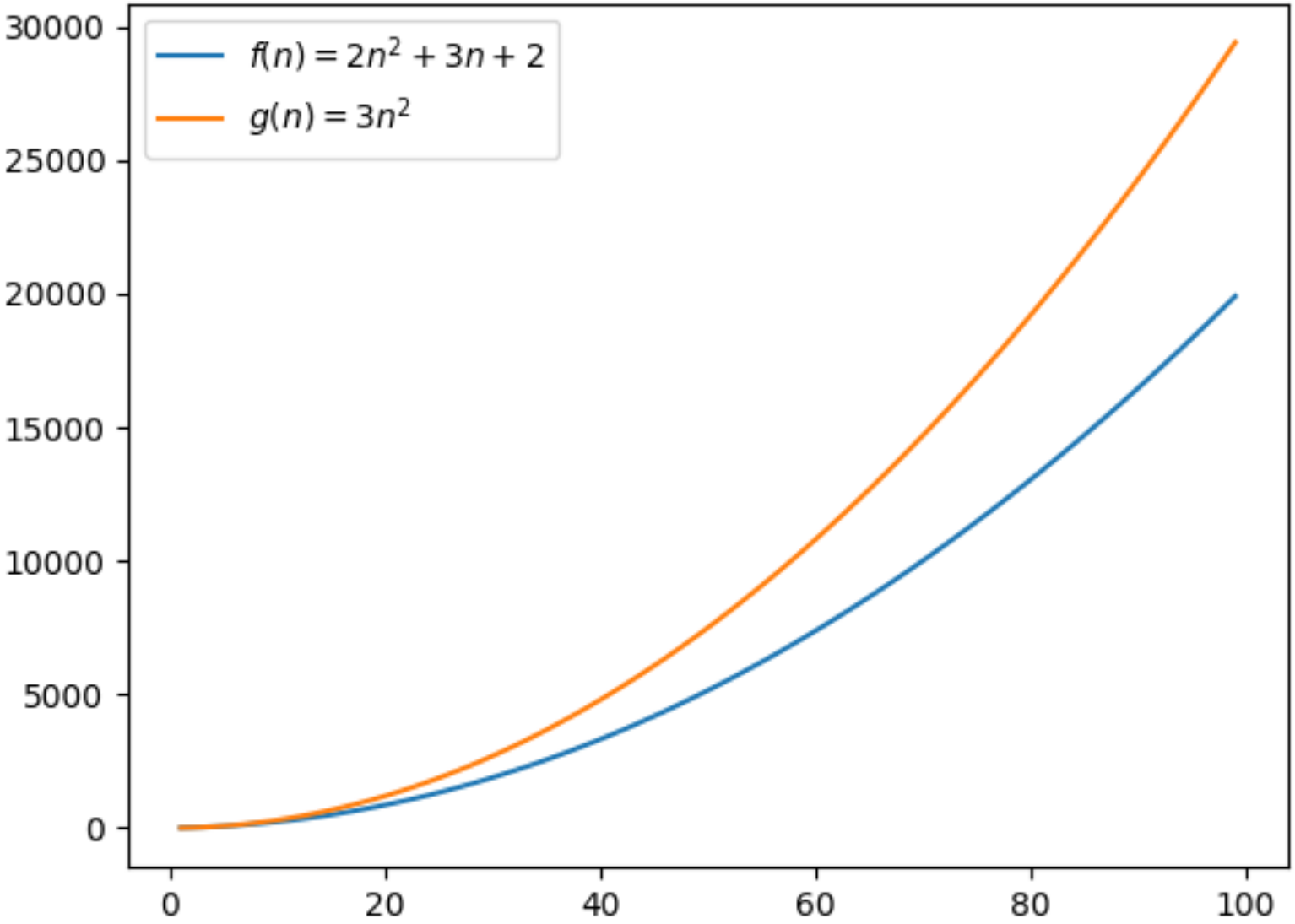
Big ‘oh’ Example - 2 (plots using matplotlib)



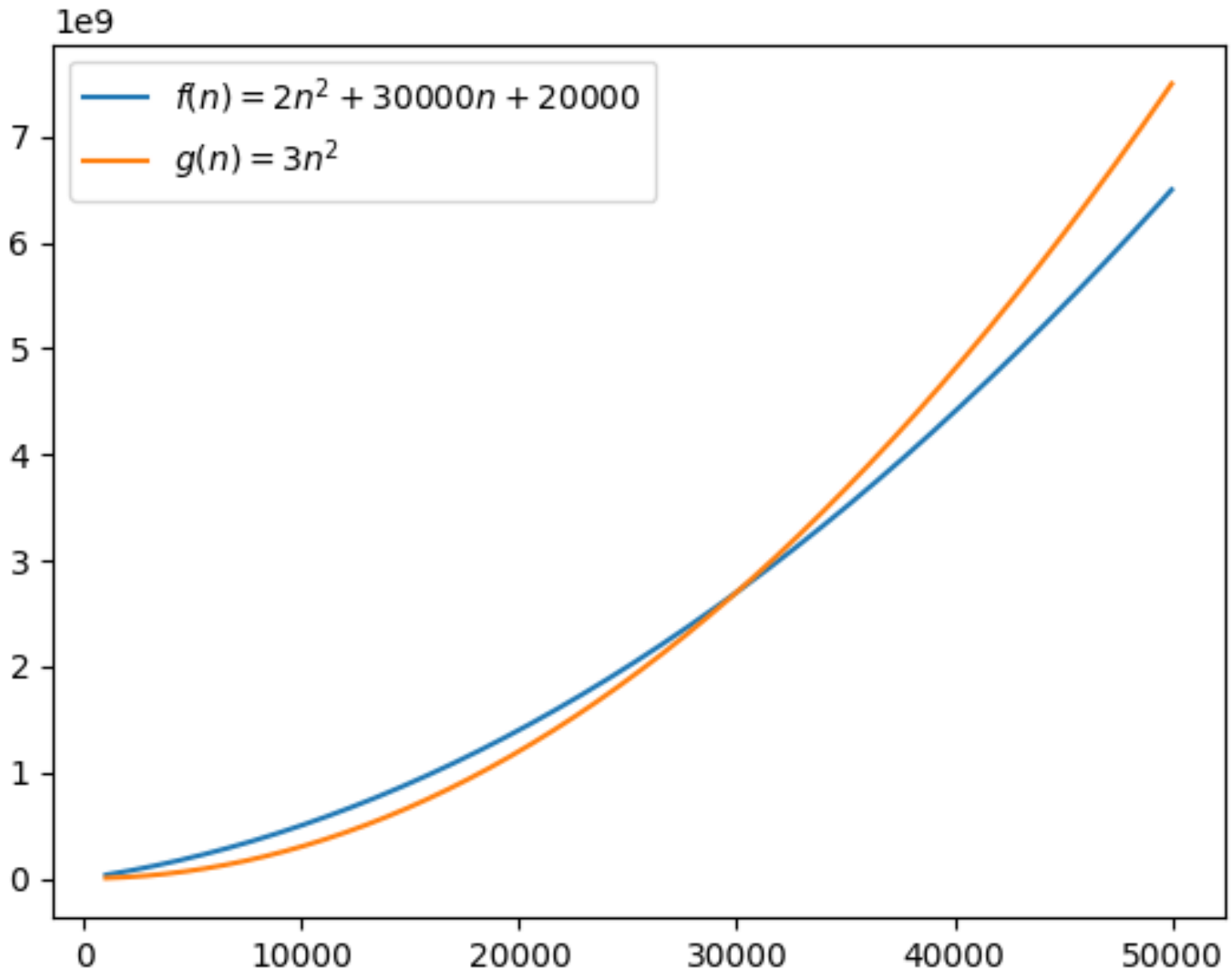
Time Complexity of Algorithms - Big 'oh' Notation(2)

- $f(n)$ is $O(g(n))$ iff there exist positive constants c and k such that $f(n) \leq c.g(n)$, for all $n \geq k$.
- Examples
  - $f(n) = 2n^2 + 3n + 2$ is $O(n^2)$, because $2n^2 + 3n + 2 \leq 3n^2$, for all $n \geq 4$ ($c = 3,\ k = 4,\ g(n) = n^2$).
  - $f(n) = 2n^2 + 30000n + 20000$ is $O(n^2)$, because $2n^2 + 30000n + 20000 \leq 3n^2$, for all $n \geq 35500$ ($c = 3,\ k = 35500,\ g(n) = n^2$). Try $c = 30000$ or $c = 50000$
  - $f(n) = n$ is NOT $O(log_2 n)$, because $n \leq c.log_2 n \Rightarrow 2^{n/c} \leq n$, is obviously false.

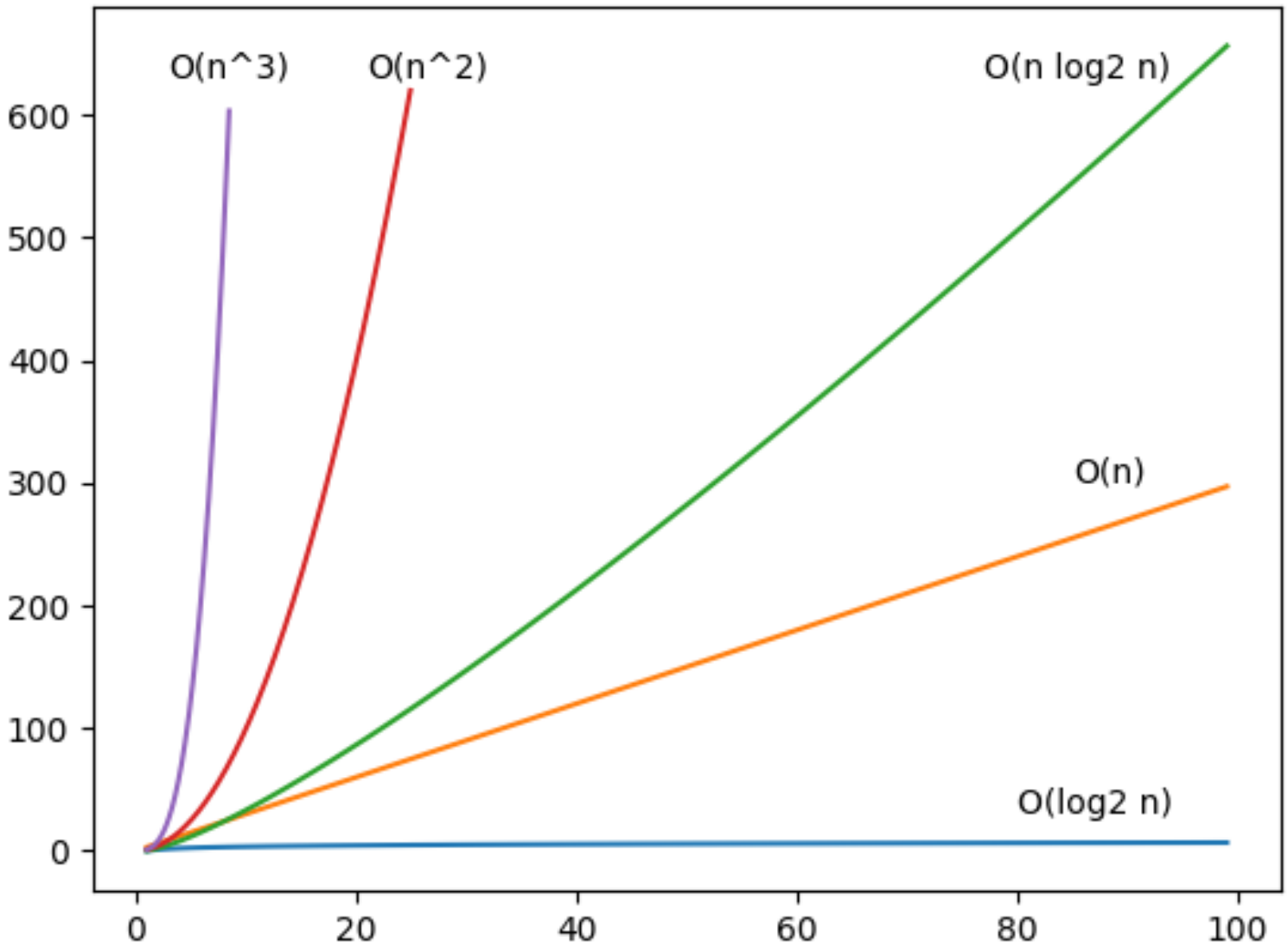
Big ‘oh’ Example - 3 (plots using matplotlib)



Big ‘oh’ Example - 4 (plots using matplotlib)



Growth of Functions (plots using matplotlib)



Time Complexity of Algorithms - Big 'oh' Notation(2)

- If an algorithm takes $O(g(n))$ time, then this is the worst case time complexity of the algorithm.
  - It is not certain that the algorithm actually takes this much time on any input.
  - It is an upper bound on the time that the algorithm takes.
  - No execution of the algorithm will take more than this much time.
- Implication - an algorithm which takes $O(log\ n)$ time is faster than an algorithm which takes $O(n)$ time, when n is sufficiently large. For small values of n, this may not be true.

Combining Functions - T1(n)+T2(n) and T1(n).T2(n)

If $T1(n)$ is $O(f(n))$ and $T2(n)$ is $O(g(n))$ then,

1. $T1(n) + T2(n)$ is $O(max(f(n), g(n))$.
   - Needed for composition.
   - Example: If $T1(n) = O(log\ n)$ and $T2(n) = O(n)$, then $T1(n) + T2(n) = O(max(log\ n, n)) = O(n)$.
2. $T1(n).T2(n)$ is $O(f(n).g(n))$.
   - Needed for loops.
   - Example: If $T1(n) = O(log\ n)$ and $T2(n) = O(n)$, then $T1(n).T2(n) = O((log\ n).n) = O(n\ log\ n)$.

Homework: Not to be submitted

1. Order the following functions by growth rate: $N, N^{0.5}, N^{1.5}, N\ log\ N, N^2, N\ log\ log\ N, N\ log^2 N, N\ log(N^2), 2/N, 2^{N/2}, 37, N^2 log\ N, N^3$. For example, we know that N^2 grows faster than N, and therefore, N precedes N^2 in the ordering. Indicate which functions grow at the same rate.
2. Prove that $log_2(n+1) = O(log\ n)$.
3. Prove or disprove: $2^{n+1} = O(2^n)$.
4. Prove or disprove: $2^{2n} = O(2^n)$.

Model of Computation (Similar to Python language)

1. A simple operation (+,-,*,/,=, etc.), a simple expression, a simple assignment, or a simple condition: $O(1)$ time.
2. A single read/print (simple data types): $O(1)$ time.
   Complete strings, lists, etc., are not included here.
3. `if/while(simple condition)`: $O(1)$ time.
4. `for var in range`: $O(1)$ time to compute the range.
5. A sequence of statements: sum of timing requirements of all statements.
6. `while` and `for` loops: O(1) + time(body)*n,
   where time(body) is the timing requirement of the body of the loop, and n is the number of iterations of the loop.
7. `if(cond) then-part elif-parts else-part`:
   maximum of timing requirements of `cond`, `then-part`, `elif-parts`, and `else-part`.
8. A function call: time required for the body of the function.

Polynomial Evaluation - Python Example 1

```
x = int(input("Give an integer value for x: "))
    # O(1)
poly = x*x+2*x+1
    # O(1)
print("Value of the polynomial x^2+2x+1 is ", poly)
    # O(1)
=============================================
x = complex(input("Give a complex value for x: "))
    # O(1)
poly = x*x+2*x+1
    # O(1)
print("Value of the polynomial x^2+2x+1 is ", poly)
    # O(1)
```

Overall time complexity =
$O(1) + O(1) + O(1) = O(max(1, 1, 1)) = O(1)$

Centigrade <-> Fahrenheit - Python Example 2

```
convert = int(input("Type 0 for F to C, and
                     type 1 for C to F: ")) # O(1)
T = float(input("Give the temp. in deg.: ")) # O(1)
if (convert == 0): # O(1)
   C = (T-32.0)*100.0/(212.0-32.0) # O(1)
   print("Corresponding temp. in deg. C is: ", C)
      # O(1) for then-part
elif (convert == 1): # O(1)
   F = T*(212.0-32.0)/100.0 + 32.0 # O(1)
   print("Corresponding temp. in deg. F is: ", F)
      # O(1) for elif-part
else:
   print("error") #O(1) for else-part
# O(1) for the whole if-elif-else statement
```

Overall time complexity =
$O(1) + O(1) + O(max(1, 1, 1)) = O(1)$

Counting Digits - Python Example 3

```
digi_count=[0]*10 # O(1)
n=int(input("Number?")) # O(1)
if n==0 : # O(1)
    digi_count[0] += 1 # O(1)
else:
    while n!=0 : # O(1), loop count = 1+log10 n
        q=n//10 # O(1)
        r=n%10 # O(1)
        n=q # O(1)
        digi_count[r]+=1 # O(1)
        print(r) # O(1)
    # O(1)+O(max(1,(1+log10 n)*1)=O(log10 n)
print(digi_count) # 10*O(1)
```

Overall time complexity =

$O(1) + O(log_{10} n) + 10 \times O(1) = O(log_{10} n) = O(log_2 n)$

Operations on Strings in Python - Time Complexity

1. *Assignment*: `MyName = "Srikant"` O(1)
2. *Copy*: `AlsoMyName = MyName` O(1)
3. *Concatenation*: `N = MyName+" Y.N."`
   `print(MyName+', How '+"are"+' you?')`
   O(total length of the strings)
4. *len*(*s*): E.g., `print(len(MyName))` O(1)
5. *Input* and *Output*:
   `s = input("Your Course Title?")`
   `print(MyName+` ` teaches `` `+s)`
   O(total length of the string)
6. *Indexing*: `s[i]` accesses i^{th} character of the string `s`. O(1)
7. *Slicing*: O(length of the slice)

String slicing in Python - Example 4

```
subject = "I am Srikant. How are you? \
How old are you?" # O(n),if subject length = n
pattern = "you"  # O(p),if pattern length = p
print(subject)  # O(n)
for i in range(0,len(subject)-len(pattern)+1):
     # O(1), loop count = n-p+1
   if (pattern==subject[i:i+len(pattern)]): # O(p)
      print(pattern, i, i+len(pattern)-1,\
           subject[i:i+len(pattern)])  # O(p)
```

Overall time complexity =
$O(n)+O(p)+O(n)+O((n-p+1)\times p) = O(n)+O(np) = O(np)$

Sort Function: Python example 5

```
def swap(num,a,b): # O(1)
    num[a],num[b] = num[b],num[a] # O(1)
def find_min(num,fr,to): # O(to-fr)
    min = num[fr]; min_index = fr # O(1)
    for i in range(fr+1,to):
    # loop count = (to-fr-1)
        if (num[i] < min): # O(1)
            min = num[i]; min_index = i # O(1)
    return min_index
def sort(num): # O(len(num)^2)
    size = len(num) #O(1)
    for i in range(0,size-1):# loop count=(size-1)
        min_index = find_min(num,i,size) # O(size-i)
        swap(num,i,min_index) # O(1)
num = [10,9,8,7,6,5,4,3,2,1]
print (num); sort(num); print (num)
```

Sort Function: Python example 5

Analysis:

`size` = n

$$
\begin{aligned}
T(n) &= c_1 + \sum_{i=0}^{size-2} (c_2(size - i)) \\
&= c_1 + \sum_{i=0}^{n-2} (c_2(n - i)) \\
&= c_1 + c_2(n + (n-1) + (n-2) + ... + 2) \\
&= O(1) + O(n^2) \\
&= O(n^2)
\end{aligned}
$$

Homework: Not to be submitted

Find the time complexity of the following program segments.

1. ```
   sum = 0
   for i in range(N):
       for j in range(N):
           sum += 1
   ```

2. ```
   sum = 0
   for i in range(N):
       for j in range(i*i):
           for k in range(j):
               sum += 1
   ```

3. Assuming n to be a power of 2, give the formula for the value of the *count* in terms of the value of n.

   ```
   count, x = 0, 2
   while (x < n):
       x = x*2
       count += 1
   print(count)
   ```

Homework (Not for Submission)

Find the time complexity of the following algorithms.

1. Computing x^n, in terms of n, given x and n as inputs.
2. You are given a word w (a string), and several alphabetic letters together as a string, p. Write a program to check whether w can be formed using the letters of p. The letters of p are not in any particular order (slicng is not needed here).
3. You are given a string of words separated by blanks. Write a program that uses slicing to build a new string of words with the following property: first two letters of each word are swapped with the last two letters of the same word. For example, “Srikant” becomes “ntikaSr”. Assume that all words in the string have at least four letters.
4. The input is given as a sequence of non-zero integers, $a, x_1, x_2, x_3, \ldots, 0$. Zero indicates end of input. Write a program to determine the number of times the number a occurs in the rest of the input.

String Search in Python - Example 6 (Self Reading)

Extracting Characters from a String - *backward*

```
s = input("Give the input string: ")
    # O(n), string length = n
print("Given string is: ",s) # O(n)
step = -1 # O(1)
while (step > -len(s)): # O(1), loop count = n
    new_s = '' # O(1)
    for i in range(len(s)-1,-1,step):
    # O(1), while-loop count = n,n-1,n-2,...,1
    # in various iterations of the while-loop
    # For simplicity, assume n for all iterations
        new_s = new_s+s[i] # O(i)
    print("Step = ",step,"New string is: ",new_s)
    # Assume this as O(n)for all iterations
    step -= 1 # O(1)
```

Overall time complexity =

$O(n)+O(n\times((1+\sum_{i=1}^{n} O(i))+O(n)) = O(n)+O(n\times(n^2+n)) = O(n^3)$

Python List Operations - Time Complexity ($|L| = n$)

`L=[1,2,3]` Assume n values	Assignment, creates a new list `L` with the given values	$O(n)$
`len(L)`	Finds the length of the list `L`	$O(1)$
`L[i]`	Accesses the i^{th} element of the list `L`	$O(1)$
`L[i]=y`	Changes the element at position `i` in the list `L` to `y`	$O(1)$
`L.append(val)`	Adds `val` at the end of the list `L`	$O(1)$
`L.insert(p,v)`	Inserts `v` at the position `p` in the list `L`	$O(n)$
`L=L1+L2`	Concatenates `L1` and `L2` and assigns the new list to `L`	$O(n)$
`L1=L2`	`L1` now points to the list `L2`	$O(1)$

Python List Operations - Time Complexity ($|L| = n$)

`L.pop(i)`	Removes and returns the element at position `i` in the list `L`	$O(n)$
`L.remove(val)`	Removes the first instance of `val` from the list `L`	$O(n)$
`L.index(val)`	Returns the index of the first instance of `val` in the list `L`	$O(n)$
`L.clear()`	Removes all the items in the list `L`	$O(1)$
`if x in L:`	Check if `x` is in `L`	$O(n)$

Dot Product: Python Example 7

The dot product of two vectors a and b is defined as: $a.b = \sum_{i=1}^{n} a_i b_i$. Assume the length of the two vectors to be n each.

```
vector1 = list(map(int,input().split())) # O(n)
vector2 = list(map(int,input().split())) # O(n)
if (len(vector1)==len(vector2)): # O(1)
    dot_product = 0 # O(1)
    for i in range(len(vector1)): # loop count = n
        dot_product += vector1[i]*vector2[i] # O(1)
    print(vector1, vector2) # O(n)
    print("Dot product = ", dot_product) # O(1)
else: # O(1)
    print("Error") # O(1)
```

Overall time complexity =
$O(n) + O(max(O(1) + O(n \times 1) + O(n), O(1)) = O(n)$

Duplicate Removal: Python Example 8 (self reading)

Given a list with duplicate elements, construct a new list without any duplicate elements (each element occurs exactly once). Assume that the length of the list is n.

```
list1 = [1,10,9,1,8,10,9,8,7,6,5,4,3,2,7,1,7] #O(n)
new = [] # O(1)
for i in list1: # O(1), loop count = n
    if i not in new:  # O(n)
        new.append(i) # O(1)
print("Given list is ", list1) # O(n)
print("List with unique elements is ", new) # O(n)
```

Overall time complexity =
$O(n) + O(1) + O(n \times max(n, 1)) + O(n) + O(n) = O(n^2)$

Searching Linked Lists - Python Example 9

```
class Node:
    def __init__(self,initdata):
        self.Element= initdata; self.next = None
# Insert code to create a linked list of n elements
element = int(input("Find what? ")) # O(1)
ptr = Head; found = False # O(1)
while ptr != None: # O(1), loop count = n
  if (ptr.Element == element): # O(1)
    found = True; break # O(1)
  else: # O(1)
    ptr = ptr.next   # O(1)
if (found): # O(1)
  print("Found ", element, ptr.Element) # O(1)
else: # O(1)
   print("Did not find ", element) # O(1)
```

The `while-loop` iterates n times in the worst case. Hence time complexity is $O(n)$.

Insertion in Linked Lists - Python Example 10

```
# Insert code to create a linked list of n elements
element = int(input("Insert after what? "))
insert_ele = int(input("New value? "))
ptr = Head; found = False
while ptr != None: #Searching as before - O(n)
  if (ptr.Element == element):
    found = True; break
  else:
    ptr = ptr.next
if (found): #Insertion of a new value
  print("Found ", element, ptr.Element)
  temp = Node(insert_ele) #O(1)
  temp.next = ptr.next; ptr.next = temp #O(1)
else:
  print("Did not find ", element)
```

While searching takes $O(n)$ time, insertion needs $O(1)$ time.

Deletion in Linked Lists - Python Example 11

```
#Insert code to create a linked list of n elements
#Delete multiple instances also
element = int(input("Delete What? "))
ptr = Head; prev = Head
while ptr != None:
  if (ptr.Element == element):
    if (ptr == Head):
      Head = ptr.next; ptr = Head; prev = Head
    else:
      prev.next = ptr.next; ptr = prev.next
  else:
    prev = ptr; ptr = ptr.next
```

The `while-loop` iterates n times in the worst case. Hence time complexity is $O(n)$. Deletion of a node needs $O(1)$ time (after search).

Homework (Not for Submission)

Estimate the time complexity of the following programs.

1. Find the second largest number in a linked list.
2. Find all the positive and negative numbers in a given range.
3. Check whether a linked list L1 is the reverse of another linked list L2.
4. Stack/queue insertion and deletion.
5. Creating a sorted linked list of numbers, reading the numbers from the input (one at a time).

The *Min-Max* Fn. - Python Example 12

Assume the length of the list as n.

```
def min_max_list(a):
      max,min = a[0],a[0] # O(1)
      for i in range(1,len(a)):
         # O(1), loop count = n
         if a[i] > max:  # O(1)
            max = a[i] # O(1)
         if a[i] < min: # O(1)
            min = a[i] # O(1)
      return [min,max] # O(1)

a = list(map(int, input("input?").split(',')))  # O(n)
print(min_max_list(a))
   # O(n) for the call, O(1) for the print
```

Overall time complexity of the function `min_max_list` =
$O(1) + O(n \times 1) + O(1) = O(n)$

Time Complexity of Recursive Programs

- Since recursive functions call themselves (directly or indirectly), their time complexity cannot be estimated using methods applicable to non-recursive functions.
- We need to use the method of *recurrence equations* or recurrences (as they are called).
- Simple recurrences can be solved by *expansion*, as in the following slides.
- Complex recurrences need more mathematical machinery (not covered in this course).

Recursive Factorial Function - Python Example 13

$factorial(1) = 1$
$factorial(n) = n * factorial(n-1)$

```
def factorial(i): # T(i)
      if(i==0) or (i==1): # O(1)
         return 1 # O(1)
         # T(1) = c1, a constant
      else: # O(1)
         return(i*factorial(i-1)) # T(i-1)+c2
   #choose some c such that c>c1 and c>c2.
   #One easy choice is c=c1+c2.

var = int(input("Number?"))
print(factorial(var)) # T(n)
```

Recurrence for the Factorial Function - Example 13

```
factorial(n) = 1 if n=1
             = n*factorial(n-1), if n>1
Recurrence equation for time is:
T(n) = c, if n=1
     = T(n-1) + c, if n>1
Expanding the recurrence equation,
T(n) = T(n-2) + c + c = T(n-2) + 2*c
     = T(n-3) + c + c + c = T(n-3) + 3*c = ...
     = T(n-(n-1)) + (n-1)*c
     = T(1) + (n-1)*c = c + (n-1)*c
     = c*n = O(n)
```

Recursive Fn. for Finding Digits - Python Example 14

```
def dcount(x): # T(x)
    if x!=0 : # O(1)
        q=x//10 # O(1)
        r=x%10 # O(1)
        digi_count[r]+=1 # O(1)
        print(r) # O(1)
        dcount(q) # T(x/10)+c

digi_count=[0]*10  # O(1)
n=int(input("Number?"))  # O(1)
if n==0 : # O(1)
    digi_count[0] += 1 # O(1)
    # T(0)=c
else: # O(1)
    dcount(n) # T(n)
```

Recurrence for the Finding Digits Fn. - Example 14

```
T(n) = k, if n=0
     = k + T(n/10), if n>0
Expanding the recurrence equation,
T(n) = k + k + T(n/100)
     = k + k + ... + k ((1+log10 n) terms)
     = k(1+log10n)  = O(log2 n)
```

Homework: Not to be submitted

Write and solve recurrence equations for the worst case time complexity of the following programs.

1. The *Tower of Hanoi* problem.
2. Recursive versions of the *min-max* function and the sorting algorithm.
3.
```
def what(n):
    if (n==1):
        return 1
    else:
        temp = what(n/2) + 1
        for i in range(n):
            temp += i
        return temp
```

Homework: Not to be submitted

Write and solve recurrence equations for the worst case time complexity of the following program segment.

3.
```
def why(n):
    if (n==1):
        return 1
    else:
        temp = why(n/2) + why(n/2)
        for i in range(n):
            temp += i
        return temp
```

Maximum Subsequence Sum Problem - Algorithm 1

Given integers $a_1, a_2, ..., a_n$, find the maximum value of $\sum_{k=i}^{j} a_k$, $i = 1, ..., n$ and $j = i, ..., n$.This value is zero if all the integers are negative.
Example sequence: -1, 5, 4, -3, -7, 2, 13, 8, -1.
Maximum subsequence sum = 23 (sum of 2, 13, 8).

```
def max_sub_sum_1(A):
    this_sum = 0; max_sum = 0; N = len(A)
    for i in range(0, N):
        this_sum = 0
        for j in range(i, N):
            this_sum += A[j]
            if (this_sum > max_sum):
                max_sum = this_sum
    return max_sum
```

Algorithm 1 is of $O(n^2)$ time complexity.

Maximum Subsequence Sum Problem - Algorithm 2

Example sequence: -1, 5, 4, -3, -7, 2, 13, 8, -1.
Maximum subsequence sum = 23 (sum of 2, 13, 8).

```
def max_sub_sum_2(A):
    this_sum = 0; max_sum = 0; N = len(A)
    for i in range(0, N):
        this_sum += A[i]
        if (this_sum > max_sum):
                max_sum = this_sum
        else:
            if (this_sum < 0):
                this_sum = 0
    return max_sum
```

Algorithm 2 is of $O(n)$ time complexity.
Homework: Modify the above algorithms to provide the indices of the max subsequence and print out the max subsequence.